Module 2: Components & Templates

Building Reusable User Interfaces with Angular 21

ANGULAR 21 FRONTEND DEVELOPMENT



Module Overview

By the end of this module, you will be able to:

1	**Standalone Components** Understand the modern Angular architecture without NgModule
2	**Component Composition** Build reusable UIs by combining smaller components
3	**Template Syntax** Write expressive Angular templates with directives and pipes
4	**Data Binding** Master interpolation, property, event, and two-way binding
5	**Lifecycle Hooks** Run code at the right moment in a component's lifetime
6	**Template Utilities** Access DOM elements and child components via ViewChild

What is a Component?

A **Component** is the fundamental building block of an Angular application. Every visible part of the UI — from a login page to a single button — is represented by a component.

Common Examples

- **Login Page**
- **Dashboard**
- **Nav Bar**
- **Employee Card**
- **Product List**
- **Footer**

Why Components?

→ Reusable

Write once, use anywhere in the app

→ Independent

Each component manages its own logic

→ Testable

Isolated units are easier to unit test

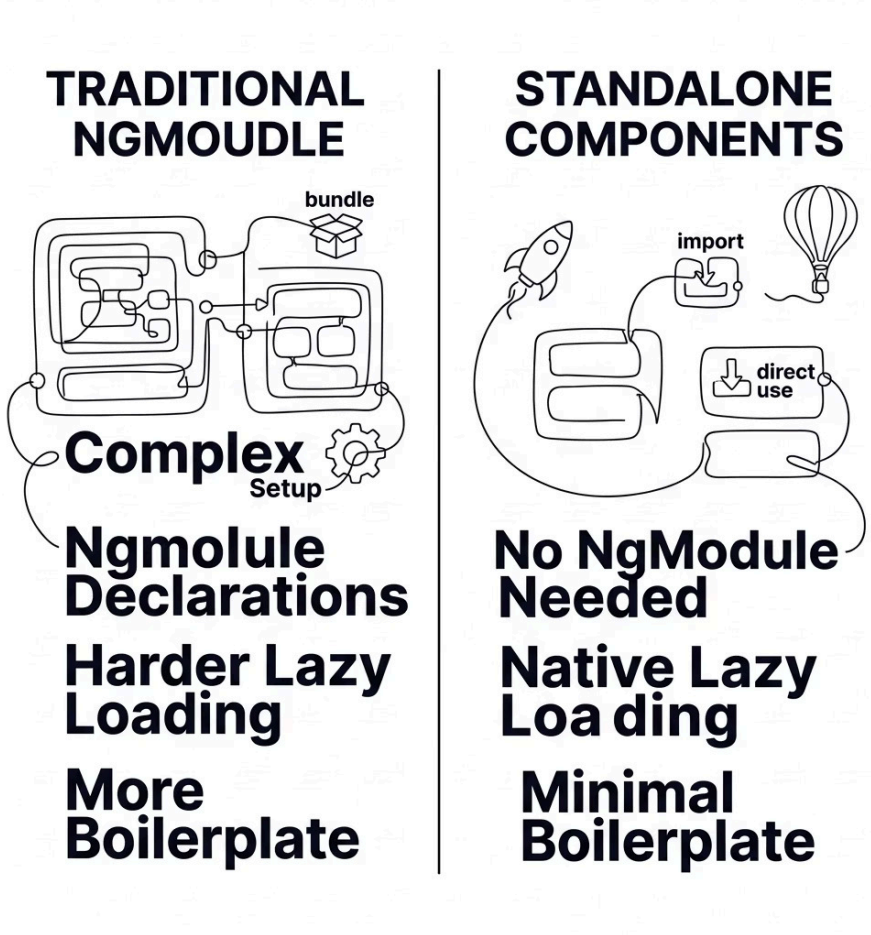
→ Maintainable

Changes stay scoped and predictable

Angular applications are built by composing multiple small, focused components.

Standalone Components

Angular 21 promotes **Standalone Components** as the default architecture, eliminating the need for `NgModule` in most applications.



Key Advantages

Less Boilerplate

No more `NgModule` wiring

Simpler Structure

Each component is self-contained

Better Lazy Loading

Load only what's needed, when needed

Easier Reuse

Import components directly where used

Creating Standalone Components

Three Ways to Create

01

Angular CLI

`ng generate component employee --standalone` — the fastest and most reliable approach

02

Manual Creation

Create the files by hand and configure the decorator yourself — useful for learning internals

03

IDE Extensions

VS Code Angular extensions scaffold components via right-click menus

Component File Structure

```
EmployeeComponent/
  ├── employee.component.ts
  ├── employee.component.html
  ├── employee.component.scss
  └── employee.component.spec.ts
```

Each component is self-contained — its own template, styles, and logic live together, making it portable and reusable across the application.

Component Metadata

Every component is configured using the `@Component` decorator. Metadata tells Angular how the component should behave and how it should be rendered.

`selector`

The HTML tag used to embed this component

`standalone`

Marks the component as standalone (`true`)

`imports`

Declares dependencies used in the template

`templateUrl`

Path to the external HTML template file

`styleUrls`

Component-scoped stylesheets

`changeDetection`

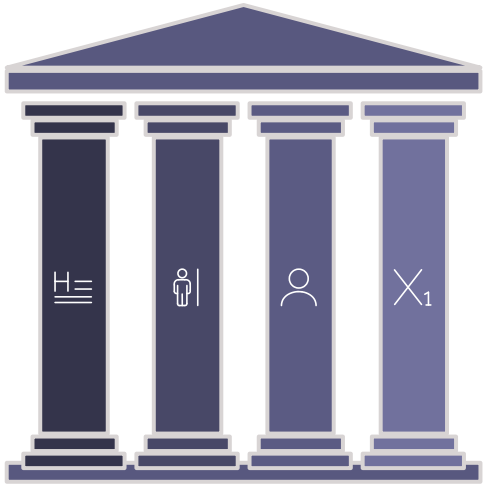
Controls how Angular detects changes (`OnPush` recommended)

Component Composition

Large applications are built by assembling smaller, reusable components into a component tree. Each piece is independent yet works together seamlessly.

Header
Top navigation and branding

Sidebar
Navigation and filters



Employee List
Contains multiple employee cards

Footer
Supplementary links and info

Each node in the tree is a standalone component with its own template, styles, and logic.

Benefits of Composition

Reusability

Build once, embed the selector anywhere

Maintainability

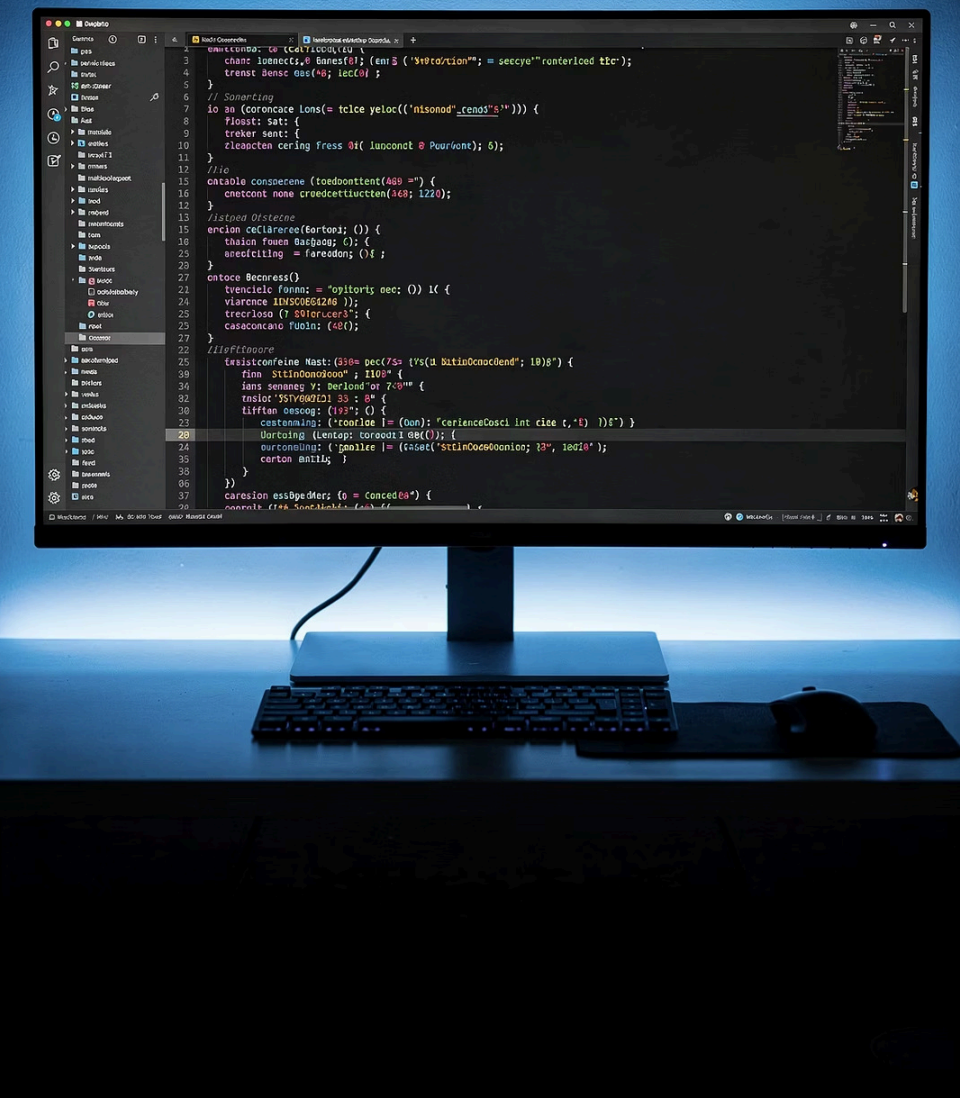
Update a single component, changes propagate everywhere

Separation of Concerns

Each component owns exactly one responsibility

Easier Testing

Test components in complete isolation



Angular Templates

Templates define how component data is displayed in the browser. They extend standard HTML with Angular-specific syntax and update automatically when application state changes.

HTML Foundation

Any valid HTML is a valid template starting point

Data Binding

Connect component data to the DOM declaratively

Pipes

Transform displayed values inline — dates, currency, uppercase

Control Flow

`@if`, `@for`, and `@switch` for conditional rendering

Data Binding in Angular

Angular provides four primary binding techniques that keep the UI synchronized with application state — without manual DOM manipulation.

Type	Syntax	Purpose
Interpolation	`{{ value }}`	Display component data as text inside the template
Property Binding	`[property]="expr"`	Dynamically set element or component properties
Event Binding	`(event)="handler()"`	Listen to user or DOM events and call component methods
Two-Way Binding	`[(ngModel)]="prop"`	Keep UI input and component data in sync simultaneously

Two-way binding combines property and event binding in a single, concise syntax often called "banana in a box."

Interpolation

What It Does

Interpolation uses the `{{ }}` double-curly-brace syntax to display component data directly inside the HTML template. Angular evaluates the expression and renders it as a string.

Common Use Cases

Key Benefits

Simple

No additional syntax or imports needed

Readable

Templates remain close to plain HTML

Auto-Updates

Re-renders whenever the bound value changes

```
<h1>Hello, {{ employeeName }}!</h1>
<p>Price: {{ product.price | currency }}</p>
```

Employee Name

Welcome Message

Product Price

Order Status

Property Binding

What It Does

Property binding uses square-bracket syntax `[property]` to dynamically set element or component properties from the component class. Data flows **one way** — from component to template.

Common Examples

Why It Matters

Dynamic UI

Element properties respond to runtime state

Better Performance

Only the bound property updates, not the whole element

One-Way Flow

Keeps data flow predictable and traceable

```
<img [src]="employee.photoUrl" />
<button [disabled]="isLoading">Save</button>
```

Image Source	Button Disabled	CSS Classes	Form Values

Event Binding

Event binding uses parenthesis syntax `(event)` to listen for user interactions and call component methods in response. It bridges user actions and application logic.



Click Events

`(click)="onSave()"` — respond to button presses and element clicks



Key Press Events

`(keyup)="onSearch($event)"` — react to keyboard input in real time



Form Submit

`(submit)="onSubmit()"` — intercept form submissions before they reach the server

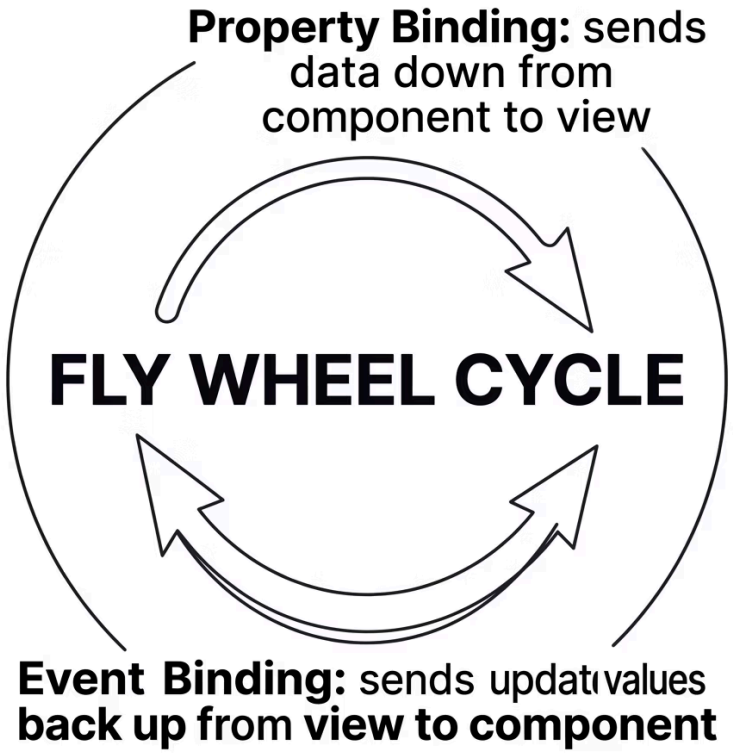


Change Events

`(change)="onToggle()"` — track dropdown, checkbox, and radio state changes

Two-Way Binding

Two-way binding combines property binding and event binding into a single concise syntax, keeping the UI and component data perfectly in sync at all times.



Common Use Cases

Forms

Input fields stay in sync with model properties automatically

Search Boxes

Query string updates as the user types

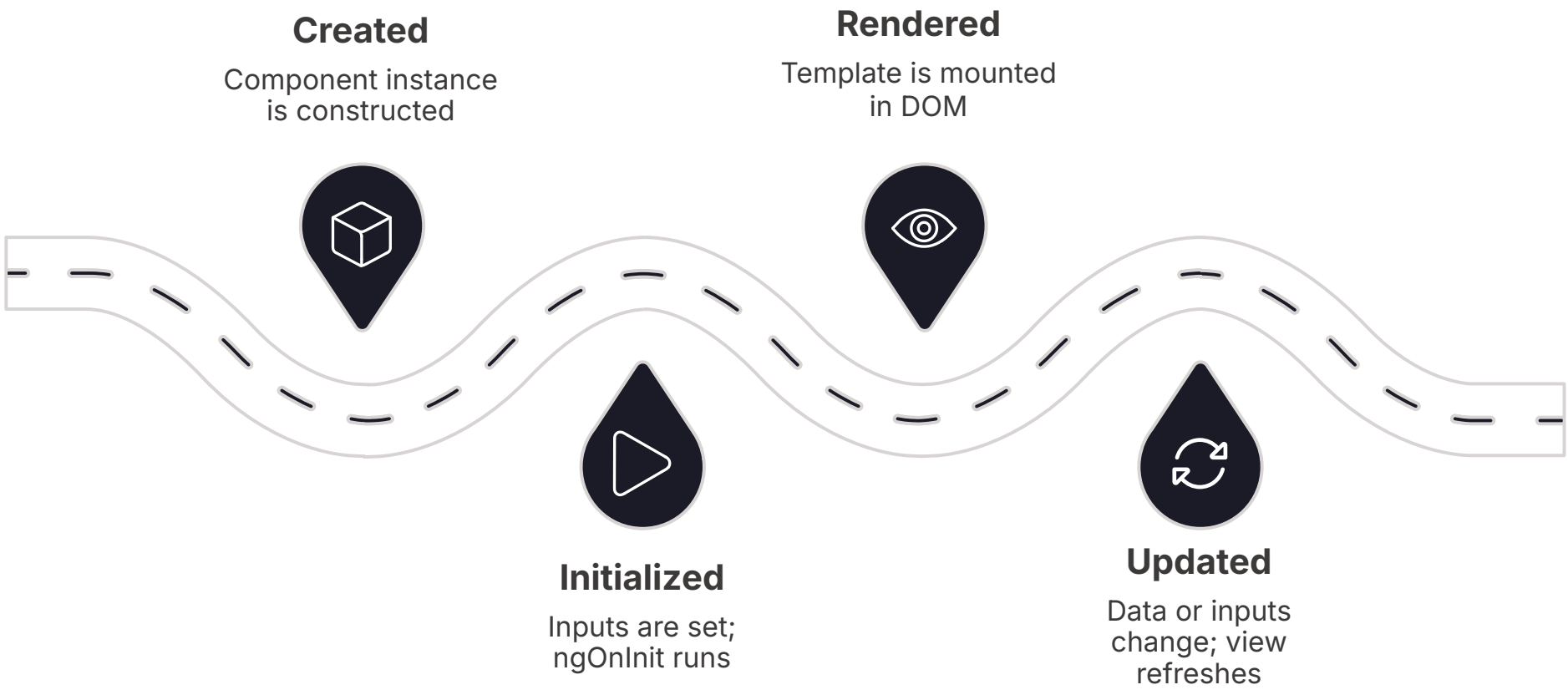
Filters & Toggles

UI state reflects component state immediately

```
<input [(ngModel)]="searchQuery" />
<p>Searching: {{ searchQuery }}</p>
```

Component Lifecycle

Every Angular component passes through a defined lifecycle — from creation and rendering to updates and final destruction. Angular provides **lifecycle hooks** so you can execute code at exactly the right moment.



Hooks give you precise control at each stage — perfect for loading data, managing subscriptions, and releasing resources cleanly.

Important Lifecycle Hooks

Hook	Purpose
`constructor()`	Dependency injection setup
`ngOnInit()`	Initialize component, fetch data
`ngOnChanges()`	Respond to `@Input()` changes
`ngAfterViewInit()`	Access the fully rendered view
`ngOnDestroy()`	Unsubscribe and release resources

Common Patterns

→ ngOnInit

Make HTTP calls to load API data on startup

→ ngOnChanges

React when parent passes new `@Input` values

→ ngAfterViewInit

Access `ViewChild` references after the DOM renders

→ ngOnDestroy

Cancel subscriptions to prevent memory leaks

Change Detection

Angular's change detection engine continuously monitors component state and updates the DOM automatically — no manual DOM manipulation required.



UI Stays in Sync

Rendered output always reflects the latest state

No Manual DOM Work

Angular handles DOM updates declaratively

OnPush Strategy

Use `ChangeDetectionStrategy.OnPush` for better performance



Template Utilities

Angular provides two powerful mechanisms for interacting with DOM elements and child components directly from the template and component class.

Template Reference Variables

Declare a `#ref` variable directly on an HTML element or directive. You can then pass it to event handlers or read its value in the template — without touching the component class.

```
<input #searchInput type="text" />
<button (click)="search(searchInput.value)">
  Go
</button>
```

View Queries

@ViewChild

Access a **single** child component or element after `ngAfterViewInit` fires

@ViewChildren

Access a **QueryList** of multiple matching child components or elements

View queries are only available after the view is fully initialized — always access them in `ngAfterViewInit()`.

Module Summary

Here's everything covered in Module 2 at a glance:

01

Standalone Components

Modern, NgModule-free component architecture

02

Component Metadata

`@Component` decorator properties and their roles

03

Composition & Templates

Building UIs from component trees with Angular template syntax

04

Data Binding

Interpolation, property, event, and two-way binding

05

Lifecycle & Change Detection

Hooks for every stage plus automatic DOM sync

06

Template Utilities

Reference variables, ViewChild, and ViewChildren



Up Next: Modern Angular Control Flow

In **Module 3**, you'll master Angular's new built-in control flow syntax — replacing legacy structural directives with cleaner, more performant alternatives.

`@if`

Conditionally render blocks with an optional `@else` branch

`@for`

Iterate over collections with a required `track` expression for performance

`@switch`

Cleanly handle multiple conditional branches in a template

MODULE 3 PREVIEW